

Lecture : 9

1. What is JSX?

JSX (JavaScript XML) is a syntax extension for JavaScript. It looks like HTML, but it lives inside your JavaScript files. It allows you to describe what the UI should look like in a way that is visually easy to understand.

- **Not valid JS:** Browsers cannot read JSX directly.
- **Declarative:** You tell React *what* you want (e.g., a button), and JSX makes that code readable.

2. What is React.createElement?

This is the actual JavaScript function that React uses to create elements. Before your code reaches the browser, a compiler (like **Babel**) transforms every line of JSX into a `React.createElement()` call.

The Syntax

`$$React.createElement(type, props, ...children)$$`

- **type:** The HTML tag (like 'div') or a React component.
- **props:** An object containing attributes (like `className` or `id`).
- **children:** Anything contained inside the element (text, other elements).

3. The Transformation: JSX vs. Plain JS

To see the difference, look at how the same piece of UI is written in both formats.

Using JSX

JavaScript

```
const element = (  
  <h1 className="title">  
    Hello, World!  
  </h1>  
);
```

Using createElement (The Compiled Result)

JavaScript

```
const element = React.createElement(  
  'h1',  
  { className: 'title' },  
  'Hello, World!'  
);
```

4. Key Differences and Rules

While JSX feels like HTML, it has a few "gotchas" because it is ultimately JavaScript:

Feature	JSX Requirement	Reason
Class Names	Use <code>className</code>	<code>class</code> is a reserved keyword in JavaScript.
Styles	Must be an object: <code>style={{ color: 'red' }}</code>	Inline styles are treated as JS objects, not strings.
Closing Tags	All tags must be closed (e.g., <code>
</code>)	JSX follows XML rules for strictness.
Expressions	Wrap in curly braces <code>{ }</code>	This tells React to evaluate the code as JavaScript.

5. Why do we use JSX?

If `React.createElement` is what the browser actually "sees," why don't we just write that?

1. **Readability:** Nesting multiple `createElement` calls becomes a "callback hell" of objects and brackets very quickly.
2. **Maintenance:** It is much easier to spot a structural error in a visual, HTML-like tree than in a series of function calls.
3. **Tooling:** Modern editors provide excellent linting and autocomplete specifically for JSX.

1. useState

When you call `useState`, it returns an array with exactly two values:

1. **The current state value:** The data you want to keep track of.
2. **The setter function:** A function used to update that data and trigger React to re-render the component.

The Syntax

JavaScript

```
const [state, setState] = useState(initialValue);
```

Note: We use **array destructuring** (the `[]` syntax) here because it's a clean way to name your state variables whatever you like.

2. How it Works in Practice

Here is a classic "Counter" example to show how the state and the setter function interact.

JavaScript

```
import React, { useState } from 'react';
```

```
function Counter() {
```

```
  // 1. Declare state variable 'count', initialized at 0
```

```
  const [count, setCount] = useState(0);
```

```
  return (
```

```
    <div>
```

```
    <p>You clicked {count} times</p>
```

```
    /* 2. Update state using the setter function */
```

```
<button onClick={() => setCount(count + 1)}>
  Click me
</button>
</div>

);
}
```

3. Important Rules of useState

State is "Read-Only"

You should never try to change the state variable directly.

- **Wrong:** `count = count + 1;`
- **Right:** `setCount(count + 1);`
React only knows it needs to update the screen if you use the **setter function**.

The Initial Value

The `initialValue` is only used during the **first render**. In every subsequent render, React ignores the initial value and uses the most recent value provided by the setter function.

Functional Updates

If your new state depends on the previous state (like a counter), it's safer to pass a function to the setter. This prevents bugs caused by asynchronous updates:

JavaScript

```
setCount(prevCount => prevCount + 1);
```

4. Handling Objects and Arrays

`useState` doesn't automatically "merge" updates if your state is an object. You have to do it manually using the spread operator (...).

Data Type	Update Pattern
Number/String	setCount(5)
Array	setItems([...items, newItem])
Object	setUser({...user, name: 'Alex'})
